

KUBERNETES (K8S) TASK - 07

Task 1: Taint a Node and Schedule a Tolerant Pod

Taint a node with `special=true:NoSchedule`. Create a pod with a toleration that matches the taint, allowing it to be scheduled on the tainted node.

Taint a Node + Toleration

Objective

Taint a node with:

`special=true:NoSchedule`

Normally, pods without a matching toleration **cannot be scheduled** on this node.

A pod with the matching toleration **can be scheduled** there.

Before starting

Check your nodes:

`kubectl get nodes -o wide`

You should see something similar to:

NAME	STATUS	ROLES	AGE
k8s-master	Ready	control-plane	...
k8s-worker1	Ready	<none>	...
k8s-worker2	Ready	<none>	...

For these exercises, I'll use **k8s-worker1** as the node we want to control.

```
ubuntu@k8s-master: ~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master    Ready     control-plane   7d4h   v1.34.11   172.31.17.127 <none>         Ubuntu 26.04.1 LTS   7.0.0-1012-aws   containerd://2.2.2
k8s-worker1    Ready     <none>         7d4h   v1.34.11   172.31.16.155 <none>         Ubuntu 26.04 LTS     7.0.0-1012-aws   containerd://2.2.2
k8s-worker2    Ready     <none>         7d3h   v1.34.11   172.31.26.51  <none>         Ubuntu 26.04 LTS     7.0.0-1012-aws   containerd://2.2.2
ubuntu@k8s-master:~$
```

Step 1: Taint the node

```
kubectl taint nodes k8s-worker1 special=true:NoSchedule
```

Expected:

```
node/k8s-worker1 tainted
```

Verify:

```
kubectl describe node k8s-worker1 | grep -i taint
```

Expected:

```
Taints: special=true:NoSchedule
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl taint nodes k8s-worker1 special=true:NoSchedule  
node/k8s-worker1 tainted  
ubuntu@k8s-master:~$ kubectl describe node k8s-worker1 | grep -i taint  
Taints:                special=true:NoSchedule  
ubuntu@k8s-master:~$
```

Step 2: Create a pod WITHOUT toleration

Create:

```
vi taint-pod.yaml
```

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: taint-test-pod  
spec:  
  containers:  
  - name: nginx  
    image: nginx
```

Save and create:

```
kubectl apply -f taint-pod.yaml
```

Check:

```
kubectl get pod taint-test-pod -o wide
```

The pod will normally be scheduled on another node, because k8s-worker1 has the NoSchedule taint.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ vi taint-pod.yaml  
ubuntu@k8s-master:~$ nano taint-pod.yaml  
ubuntu@k8s-master:~$ kubectl apply -f taint-pod.yaml  
pod/taint-test-pod created  
ubuntu@k8s-master:~$ kubectl get pod taint-test-pod -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES  
taint-test-pod 1/1     Running   0           20s   192.168.126.59 k8s-worker2 <none>          <none>
```

Step 3: Create a pod WITH toleration

Create:

vi tolerant-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: tolerant-pod  
spec:  
  tolerations:  
    - key: "special"  
      operator: "Equal"  
      value: "true"  
      effect: "NoSchedule"  
  
  containers:  
    - name: nginx  
      image: nginx
```

Create:

kubectl apply -f tolerant-pod.yaml

Check:

kubectl get pod tolerant-pod -o wide

Important point

The toleration means:

Pod:

"I tolerate special=true:NoSchedule"

Node:

"I have special=true:NoSchedule"

Result:

Pod is allowed to run on that node.

Toleration does not force the pod onto that node. It only makes the pod eligible. If you specifically want to force it there, combine the toleration with nodeSelector or node affinity.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano tolerant-pod.yaml  
ubuntu@k8s-master:~$ kubectl apply -f tolerant-pod.yaml  
pod/tolerant-pod created  
ubuntu@k8s-master:~$ kubectl get pod tolerant-pod -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES  
tolerant-pod  1/1     Running   0           7s    192.168.194.90 k8s-worker1 <none>          <none>  
ubuntu@k8s-master:~$
```

Task 2: Use NodeSelector to Schedule a Pod on a Specific Node Label a node with env=dev. Create a pod with a nodeSelector that schedules it only on the node labeled env=dev.

NodeSelector

Objective

Label a node:

env=dev

Then configure the pod with:

nodeSelector:

env: dev

This means the scheduler can place the pod **only on nodes having env=dev**.

Step 1: Add the label

kubectl label nodes k8s-worker1 env=dev

Expected:

node/k8s-worker1 labeled

Verify:

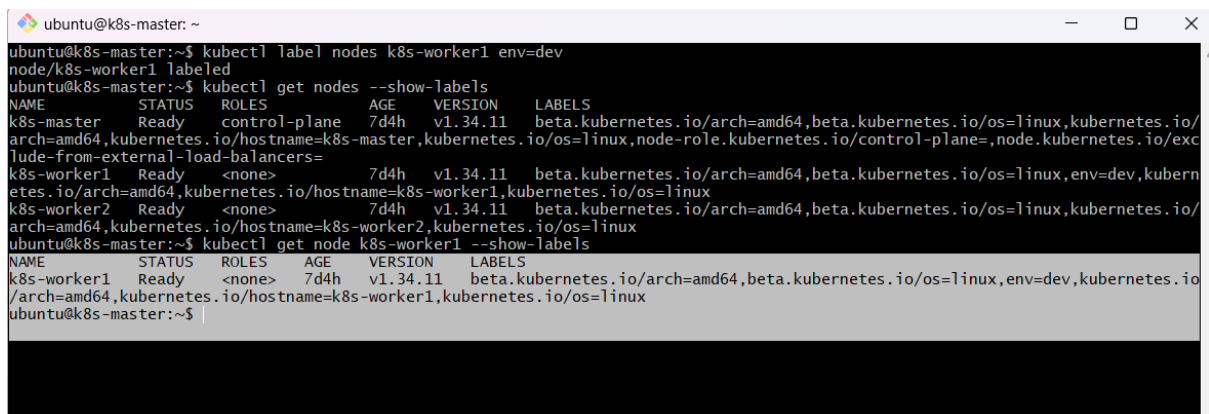
kubectl get nodes --show-labels

Or:

kubectl get node k8s-worker1 --show-labels

You should find:

env=dev



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl label nodes k8s-worker1 env=dev  
node/k8s-worker1 labeled  
ubuntu@k8s-master:~$ kubectl get nodes --show-labels  
NAME          STATUS    ROLES    AGE   VERSION   LABELS  
k8s-master    Ready     control-plane  7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=node.kubernetes.io/exclude-from-external-load-balancers=  
k8s-worker1   Ready     <none>    7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=dev,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux  
k8s-worker2   Ready     <none>    7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker2,kubernetes.io/os=linux  
ubuntu@k8s-master:~$ kubectl get node k8s-worker1 --show-labels  
NAME          STATUS    ROLES    AGE   VERSION   LABELS  
k8s-worker1   Ready     <none>    7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=dev,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux  
ubuntu@k8s-master:~$
```

Step 2: Create the pod

vi nodeselector-pod.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: node-selector-pod  
spec:  
  nodeSelector:  
    env: dev  
  
  containers:  
  - name: nginx  
    image: nginx
```

Create:

```
kubectl apply -f nodeslector-pod.yaml
```

Check:

```
kubectl get pod node-selector-pod -o wide
```

Expected:

NAME	READY	STATUS	NODE
node-selector-pod	1/1	Running	k8s-worker1

Because:

k8s-worker1

|

+---- env=dev

|

↓

nodeSelector: env=dev

|

↓

Pod scheduled

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get pod node-selector-pod -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES  
node-selector-pod 1/1     Running   0           8m23s 192.168.194.89 k8s-worker1   <none>           <none>
```

Task 3: Use Node Affinity with Soft Scheduling
Label a node with env=test. Create a pod with PreferredDuringSchedulingIgnoredDuringExecution node affinity, preferring to schedule it on a node labeled env=test. Remove the label and verify the pod continues to run.

Objective

We want Kubernetes to **prefer** a node with:

env=test

But it is **not mandatory**.

This is called:

PreferredDuringSchedulingIgnoredDuringExecution

Step 1: Label the node

First remove the old env=dev label:

```
kubectl label node k8s-worker1 env-
```

Now add:

```
kubectl label node k8s-worker1 env=test
```

Verify:

```
kubectl get node k8s-worker1 --show-labels
```

You should see:

env=test

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl label node k8s-worker1 env-  
node/k8s-worker1 unlabeled  
ubuntu@k8s-master:~$ kubectl label node k8s-worker1 env=test  
node/k8s-worker1 labeled  
ubuntu@k8s-master:~$ kubectl get node k8s-worker1 --show-labels  
NAME          STATUS    ROLES    AGE   VERSION   LABELS  
k8s-worker1   Ready     <none>    7d4h  v1.34.11  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=test,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux  
ubuntu@k8s-master:~$
```

Step 2: Create the soft-affinity pod

vi preferred-affinity.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 preferred-affinity.yaml  
apiVersion: v1  
kind: Pod  
metadata:  
  name: preferred-affinity-pod  
spec:  
  affinity:  
    nodeAffinity:  
      preferredDuringSchedulingIgnoredDuringExecution:  
      - weight: 100  
        preference:  
          matchExpressions:  
          - key: env  
            operator: In  
            values:  
            - test  
  containers:  
  - name: nginx  
    image: nginx  
[Wrote 100 lines]
```

Create:

kubectl apply -f preferred-affinity.yaml

Check:

kubectl get pod preferred-affinity-pod -o wide

Because k8s-worker1 has:

env=test

the scheduler **prefers** that node.

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl apply -f preferred-affinity.yaml  
pod/preferred-affinity-pod created  
ubuntu@k8s-master:~$ kubectl get pod preferred-affinity-pod -o wide  
NAME             READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES  
preferred-affinity-pod 1/1     Running   0          7s    192.168.194.85 k8s-worker1 <none>          <none>
```

Step 3: Remove the label

Now remove:

```
kubectl label node k8s-worker1 env-
```

Verify:

```
kubectl get node k8s-worker1 --show-labels
```

The env=test label is gone.

Now check the pod:

```
kubectl get pod preferred-affinity-pod -o wide
```

It should **continue running**.

Why?

The important word is:

IgnoredDuringExecution

The rule is considered during scheduling.

Once the pod has been scheduled, removing the label does **not** automatically evict the pod.

So:

Before scheduling:

env=test

↓

Pod prefers k8s-worker1

↓

Pod scheduled

After scheduling:

env=test removed

↓

Pod continues running

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl label node k8s-worker1 env-  
node/k8s-worker1 unlabeled  
ubuntu@k8s-master:~$ kubectl get node k8s-worker1 --show-labels  
NAME          STATUS    ROLES    AGE   VERSION   LABELS  
k8s-worker1   Ready    <none>   7d4h  v1.34.11  beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux  
ubuntu@k8s-master:~$ kubectl get pod preferred-affinity-pod -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES  
preferred-affinity-pod  1/1     Running   0          2m25s  192.168.194.85  k8s-worker1   <none>           <none>
```

Task 4: Implement Node Affinity with Hard Scheduling Create a pod with `RequiredDuringSchedulingIgnoredDuringExecution` node affinity, ensuring it will only be scheduled on a node labeled `env=prod`. Verify the pod cannot be scheduled if no node has the `env=prod` label.

Required Node Affinity — Hard Scheduling

Objective

Create a pod that **must** run on a node having:

`env=prod`

We use:

requiredDuringSchedulingIgnoredDuringExecution

Step 1: Make sure no node has env=prod

Check:

kubectl get nodes --show-labels

If you have:

env=prod

remove it:

kubectl label node k8s-worker1 env-

and, if necessary:

kubectl label node k8s-worker2 env-

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl get nodes --show-labels  
NAME          STATUS    ROLES    AGE   VERSION   LABELS  
k8s-master    Ready     control-plane  7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exclude-from-external-load-balancers=  
k8s-worker1    Ready     <none>      7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux  
k8s-worker2    Ready     <none>      7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/arch=amd64,kubernetes.io/hostname=k8s-worker2,kubernetes.io/os=linux  
ubuntu@k8s-master:~$ kubectl label node k8s-worker1 env-  
label "env" not found.  
node/k8s-worker1 not labeled  
ubuntu@k8s-master:~$ kubectl label node k8s-worker2 env-  
label "env" not found.  
node/k8s-worker2 not labeled  
ubuntu@k8s-master:~$ |
```

Step 2: Create the hard-affinity pod

vi required-affinity.yaml

```
ubuntu@k8s-master: ~  
GNU nano 8.7.1 requ  
apiVersion: v1  
kind: Pod  
metadata:  
  name: required-affinity-pod  
spec:  
  affinity:  
    nodeAffinity:  
      requiredDuringSchedulingIgnoredDuringExecution:  
        nodeSelectorTerms:  
          - matchExpressions:  
            - key: env  
              operator: In  
              values:  
                - prod  
  
  containers:  
    - name: nginx  
      image: nginx
```

Create:

```
kubectl apply -f required-affinity.yaml
```

Check:

```
kubectl get pod required-affinity-pod
```

You should see something similar to:

NAME	READY	STATUS	RESTARTS
------	-------	--------	----------

required-affinity-pod	0/1	Pending	0
-----------------------	-----	---------	---

Why?

Because there is no node with:

env=prod

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano required-affinity.yaml  
ubuntu@k8s-master:~$ kubectl apply -f required-affinity.yaml  
pod/required-affinity-pod created  
ubuntu@k8s-master:~$ kubectl get pod required-affinity-pod  
NAME                READY   STATUS    RESTARTS   AGE  
required-affinity-pod 0/1     Pending   0           7s  
ubuntu@k8s-master:~$ |
```

Step 3: Look at the scheduling reason

Run:

```
kubectl describe pod required-affinity-pod
```

Go to the bottom and look at:

Events:

You should see a scheduling message indicating that no node satisfies the pod's node affinity.

This is the key difference:

Preferred

preferredDuringScheduling...

Means:

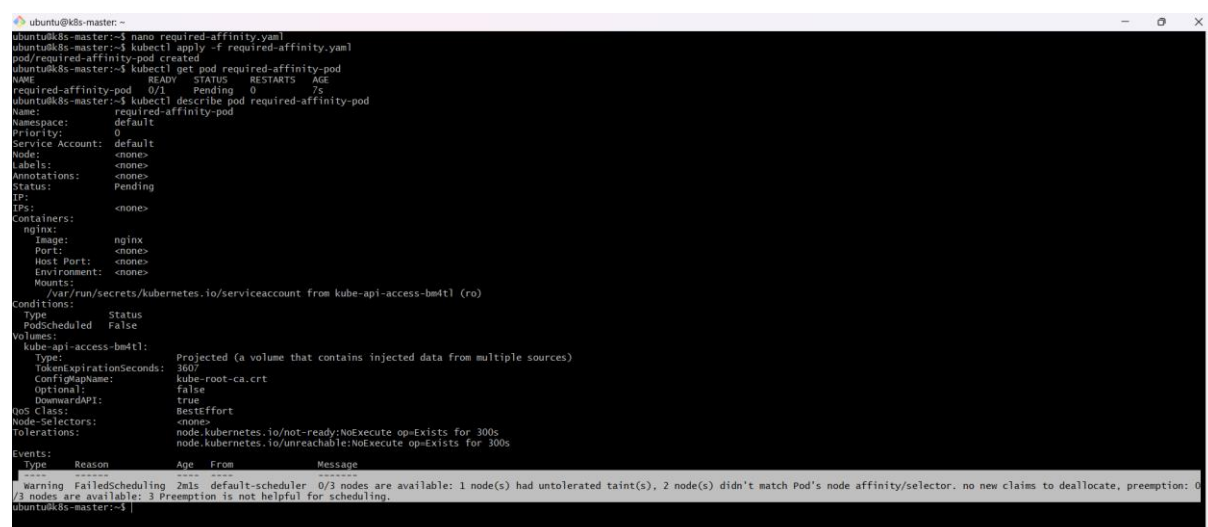
"I would like this node, but another node is okay."

Required

requiredDuringScheduling...

Means:

"I must have a node matching this rule."



```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ nano required-affinity.yaml  
ubuntu@k8s-master:~$ kubectl apply -f required-affinity.yaml  
pod/required-affinity-pod created  
ubuntu@k8s-master:~$ kubectl get pod required-affinity-pod  
NAME          READY   STATUS    RESTARTS   AGE  
required-affinity-pod  0/1     Pending   0          7s  
ubuntu@k8s-master:~$ kubectl describe pod required-affinity-pod  
Name:          required-affinity-pod  
Namespace:     default  
Priority:       0  
Service Account: default  
Node:          <none>  
Labels:        <none>  
Annotations:   <none>  
Status:        Pending  
IP:            <none>  
IPs:           <none>  
Containers:  
  nginx:  
    Image:      nginx  
    Port:       <none>  
    Host Port:  <none>  
    Environment: <none>  
    Mounts:  
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-bm4tl (ro)  
Conditions:  
  Type      Status  
  PodScheduled  False  
Volumes:  
  kube-api-access-bm4tl:  
    Type:              Projected (a volume that contains injected data from multiple sources)  
    TokenExpirationSeconds: 3607  
    ConfigMapName:       kube-root-ca.crt  
    Optional:            false  
    DownwardAPI:         true  
QoS Class:           BestEffort  
Node-Selectors:      <none>  
Tolerations:         node.kubernetes.io/not-ready:NoExecute op=Exists for 300s  
                     node.kubernetes.io/unreachable:NoExecute op=Exists for 300s  
Events:  
  Type      Reason      Age    From          Message  
  ----      -  
Warning    FailedScheduling 2m1s  default-scheduler  0/3 nodes are available: 1 node(s) had untolerated taint(s), 2 node(s) didn't match Pod's node affinity/selector. no new claims to deallocate, preemption: 0/3 nodes are available: 3 Preemption is not helpful for scheduling.  
ubuntu@k8s-master:~$
```

Step 4: Add env=prod

Now label worker1:

```
kubectl label node k8s-worker1 env=prod
```

```
ubuntu@k8s-master: ~$ kubectl label node k8s-worker1 env=prod
node/k8s-worker1 labeled
ubuntu@k8s-master: ~$ kubectl get nodes --show-labels
NAME          STATUS    ROLES    AGE   VERSION   LABELS
k8s-master    Ready     control-plane  7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/
arch=amd64,kubernetes.io/hostname=k8s-master,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=,node.kubernetes.io/exc
lude-from-external-load-balancers=
k8s-worker1   Ready     <none>     7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,env=prod,kuber
netes.io/arch=amd64,kubernetes.io/hostname=k8s-worker1,kubernetes.io/os=linux
k8s-worker2   Ready     <none>     7d4h   v1.34.11   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io/
arch=amd64,kubernetes.io/hostname=k8s-worker2,kubernetes.io/os=linux
ubuntu@k8s-master: ~$ kubectl get pod required-affinity-pod -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
required-affinity-pod  1/1     Running    0           4m35s  192.168.194.88  k8s-worker1   <none>           <none>
```

Task 5: Taint a Node and Use NoExecute with Toleration Seconds Taint a node with `special=true:NoExecute`. Create a pod with a `tolerationSeconds` field (e.g., 60 seconds) and observe it gets evicted after 60 seconds on the tainted node.

NoExecute + tolerationSeconds

This is an important exercise because NoExecute behaves differently from NoSchedule.

Objective

Taint a node:

`special=true:NoExecute`

Create a pod that tolerates this taint for:

60 seconds

After the toleration expires, Kubernetes can evict the pod.

Important: Clean up old taints first

Before this task, check:

```
kubectl describe node k8s-worker1 | grep -i taint
```

If you still have:

```
special=true:NoSchedule
```

remove it:

```
kubectl taint nodes k8s-worker1 special=true:NoSchedule-
```

Verify:

```
kubectl describe node k8s-worker1 | grep -i taint
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl describe node k8s-worker1 | grep -i taint  
Taints:          <none>  
ubuntu@k8s-master:~$ kubectl taint nodes k8s-worker1 special=true:NoSchedule-  
error: taint "special:NoSchedule" not found  
ubuntu@k8s-master:~$ kubectl describe node k8s-worker1 | grep -i taint  
Taints:          <none>  
ubuntu@k8s-master:~$ |
```

Step 1: Add the NoExecute taint

```
kubectl taint nodes k8s-worker1 special=true:NoExecute
```

Verify:

```
kubectl describe node k8s-worker1 | grep -i taint
```

Expected:

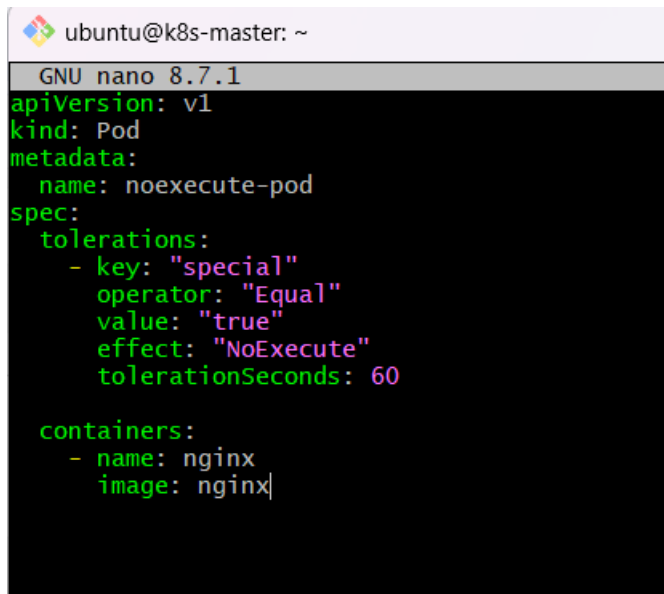
```
Taints: special=true:NoExecute
```

```
ubuntu@k8s-master: ~  
ubuntu@k8s-master:~$ kubectl taint nodes k8s-worker1 special=true:NoExecute  
node/k8s-worker1 tainted  
ubuntu@k8s-master:~$ kubectl describe node k8s-worker1 | grep -i taint  
Taints:          special=true:NoExecute  
ubuntu@k8s-master:~$ |
```

Step 2: Create the pod

Create:

vi noexecute-pod.yaml



```
ubuntu@k8s-master: ~  
GNU nano 8.7.1  
apiVersion: v1  
kind: Pod  
metadata:  
  name: noexecute-pod  
spec:  
  tolerations:  
    - key: "special"  
      operator: "Equal"  
      value: "true"  
      effect: "NoExecute"  
      tolerationSeconds: 60  
  
  containers:  
    - name: nginx  
      image: nginx
```

Create it:

kubectl apply -f noexecute-pod.yaml

Step 3: Check the pod

kubectl get pod noexecute-pod -o wide

You may notice that it isn't necessarily running on k8s-worker1.

Why?

Because, just like a NoSchedule toleration, the toleration **allows** the pod onto the node but doesn't force it there.

For this particular exercise, we want to ensure it is on k8s-worker1.

Therefore, add a node selector.

First label the node:

kubectl label node k8s-worker1 special-node=true

Then use this YAML:

apiVersion: v1

kind: Pod

metadata:

name: noexecute-pod

spec:

nodeSelector:

special-node: "true"

tolerations:

- key: "special"

operator: "Equal"

value: "true"

effect: "NoExecute"

tolerationSeconds: 60

containers:

- name: nginx

image: nginx

If the previous pod exists:

kubectl delete pod noexecute-pod

Then:

kubectl apply -f noexecute-pod.yaml

Check:

kubectl get pod noexecute-pod -o wide

You should see:

NAME	READY	STATUS	NODE
noexecute-pod	1/1	Running	k8s-worker1

Step 4: Observe the 60-second behavior

Run:

```
kubectl get pod noexecute-pod -o wide -w
```

The pod should initially be:

Running

After approximately **60 seconds**, the toleration expires and the pod is eligible for eviction from the tainted node.

Depending on scheduling conditions and pod state, you'll observe it transition away from running/being removed.

You can also check:

```
kubectl describe pod noexecute-pod
```

Look at:

Events:

```
ubuntu@k8s-master: ~
ubuntu@k8s-master:~$ nano noexecute-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f noexecute-pod.yaml
pod/noexecute-pod created
ubuntu@k8s-master:~$ kubectl get pod noexecute-pod -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
noexecute-pod 1/1 Running 0 13s 192.168.194.79 k8s-worker1 <none> <none>
ubuntu@k8s-master:~$ kubectl get pod noexecute-pod -o wide -w
Error from server (NotFound): pods "noexecute-pod" not found
ubuntu@k8s-master:~$ kubectl describe pod noexecute-pod
Error from server (NotFound): pods "noexecute-pod" not found
ubuntu@k8s-master:~$ kubectl apply -f noexecute-pod.yaml
pod/noexecute-pod created
ubuntu@k8s-master:~$ kubectl get pod noexecute-pod -o wide -w
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
noexecute-pod 1/1 Running 0 12s 192.168.194.78 k8s-worker1 <none> <none>
noexecute-pod 1/1 Running 0 60s 192.168.194.78 k8s-worker1 <none> <none>
noexecute-pod 1/1 Terminating 0 60s 192.168.194.78 k8s-worker1 <none> <none>
noexecute-pod 1/1 Terminating 0 60s 192.168.194.78 k8s-worker1 <none> <none>
noexecute-pod 1/1 Terminating 0 60s 192.168.194.78 k8s-worker1 <none> <none>
noexecute-pod 0/1 Completed 0 60s 192.168.194.78 k8s-worker1 <none> <none>
noexecute-pod 0/1 Completed 0 60s 192.168.194.78 k8s-worker1 <none> <none>
noexecute-pod 0/1 Completed 0 60s 192.168.194.78 k8s-worker1 <none> <none>
```

Very Important Difference: NoSchedule vs NoExecute

Remember this for your interview/seminar:

Taint	Meaning
NoSchedule	New pods without toleration won't be scheduled onto the node
PreferNoSchedule	Kubernetes tries to avoid scheduling there
NoExecute	New pods without toleration aren't scheduled, and existing pods without adequate toleration can be evicted

And:

tolerationSeconds: 60

means approximately:

"I can tolerate this NoExecute taint for 60 seconds."